



P2P KOMUNIKAČNÍ APLIKACE

Ročníková práce

Autor	Jakub Adam
Obor	Informační technologie
Vedoucí práce	Ing. Tomáš Kazda
Školní rok	2025/2026
Počet stran	25
Počet slov	2822

Anotace

Práce se zabývá návrhem a implementací P2P komunikační aplikace určené pro šifrovanou textovou komunikaci uživatelů. V tomto dokumentu je popsána architektura systému, princip navázání přímého spojení mezi klienty a zajištění bezpečného přenosu dat. Dále se práce věnuje návrhu uživatelského rozhraní, testování běžnými uživateli a následné optimalizaci její funkčnosti. Výsledkem je funkční aplikace pro bezpečnou a efektivní komunikaci.

Summary

This thesis focuses on the design and implementation of a P2P communication application intended for encrypted text communication between users. This document describes the system architecture, the principle of establishing direct connections between clients, and ensuring secure data transmission. Furthermore, the thesis addresses the design of the user interface, testing by regular users, and the subsequent optimization of its functionality. The result is a functional application for secure and efficient communication.

Čestné prohlášení

Prohlašuji, že jsem tuto práci vypracoval samostatně. V práci jsem použil pouze zdroje uvedené v seznamu literatury a všechny převzaté části, včetně textů, obrázků, kódů a výstupů umělé inteligence, jsou řádně označeny a citovány.

V Liberci dne 15.05.2026

.....
Jakub Adam

Obsah

Úvod.....	1
1 Analýza existujících řešení	2
1.1 Centralizovaná architektura	2
1.2 P2P architektura.....	2
1.3 Hybridní architektura.....	2
2 Architektura aplikace	3
2.1 Frontend aplikace	3
2.2 Backend aplikace	3
2.3 Databáze	3
2.4 Diagram architektury	3
2.4.1 Databázové modely	4
2.5 P2P Komunikace.....	4
2.6 Průběh navázání P2P spojení	5
2.6.1 NAT Traversal.....	6
3 Požadavky aplikace	7
3.1 Registrace a přihlášení uživatelů	7
3.2 Správa uživatelského účtu	7
3.3 Komunikace mezi uživateli.....	7
3.4 Bezpečnost komunikace	7
3.5 Spojení a dostupnost.....	7
3.6 LetsTalk! funkce.....	7
3.7 Nedávné zprávy	8
3.8 Seznam online uživatelů.....	8
3.9 Design a použitelnost.....	8
4 Realizace	9
4.1 Návrh uživatelského rozhraní.....	9
4.1.1 Intuitivní prostředí a identita.....	9
4.2 Vývoj backendu.....	9
4.2.1 Fallback server	9
4.2.2 Zpracování zpráv.....	10
4.2.3 WebSocket broadcast	10
4.3 Vývoj frontendu	11

4.3.1	Struktura aplikace.....	11
4.3.2	Správa stavů.....	11
4.3.3	Správa konverzací.....	11
4.4	API endpointy.....	11
4.5	Nasazení aplikace.....	12
5	Testování.....	13
5.1.1	Zpracování zpětné vazby od uživatelů	13
	Závěr.....	15
	Seznam zkratk a odborných výrazů.....	16
	Seznam obrázků.....	18
	Reference.....	19
A.	Seznam příložených souborů	I

Úvod

S rozvojem digitální komunikace rostou také požadavky na bezpečnost a ochranu soukromí uživatelů. Většina současných komunikačních aplikací využívá centralizovanou architekturu, při které jsou data ukládána na serverech poskytovatele služby. To zvyšuje riziko úniku dat a zároveň nutí uživatele důvěřovat provozovateli aplikace.

Alternativou je P2P komunikace, která umožňuje přímé spojení mezi uživateli bez nutnosti přenosu dat přes centrální server. Tento přístup nabízí větší soukromí komunikace, ale zároveň přináší technické výzvy při navazování spojení mezi zařízeními v různých sítích.

Cílem této práce je návrh a implementace P2P komunikační aplikace zaměřené na bezpečnou textovou komunikaci mezi uživateli. Součástí práce je návrh uživatelského rozhraní, implementace real-time komunikace a následné testování aplikace v běžném provozu.

1 Analýza existujících řešení

1.1 Centralizovaná architektura

Centralizovanou architekturu využívá většina dnešních komunikačních aplikací. Komunikace probíhá přes centrální server, který zajišťuje správu uživatelů i ukládání zpráv. Výhodou je stabilita systému, nevýhodou větší závislost na serveru a nižší soukromí uživatelů.

1.2 P2P architektura

P2P architektura umožňuje přímou komunikaci mezi uživateli bez přenosu dat přes centrální server. Výhodou je větší soukromí komunikace, nevýhodou složitější navazování spojení mezi zařízeními v různých sítích a zachování přívětivého uživatelského rozhraní.

1.3 Hybridní architektura

Hybridní architektura kombinuje vlastnosti centralizované i P2P komunikace. Část dat je spravována centrálním serverem, zatímco samotná komunikace probíhá přímo mezi uživateli. Výhodou je zachování bezpečnosti komunikace a zlepšení komfortu pro uživatele. Nevýhodou zůstává technická náročnost implementace systému.

2 Architektura aplikace

2.1 Frontend aplikace

Frontend aplikace je vytvořen pomocí React v kombinaci s TypeScript. Pro rychlé vývojové prostředí byl zvolen Vite. Pro jednoduché UI ikony je využita knihovna Lucide React. Navigace v aplikaci je řešena pomocí React Router a uživatelské rozhraní využívá ikony z knihovny Lucide React.

Frontend komunikuje s backendem prostřednictvím Express API. Část validace dat na straně klienta zajišťuje knihovna Zod.

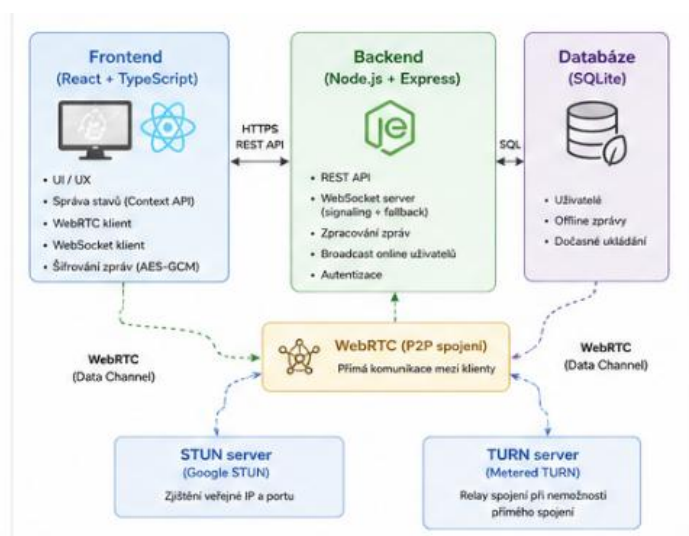
2.2 Backend aplikace

Backend je implementován pomocí runtime prostředí Node.js, které umožňuje běh JavaScriptu na serverové straně. Server zajišťuje logiku aplikace, správu uživatelů, real-time komunikaci přes WebSocket a poskytování frontendové části aplikace.

2.3 Databáze

Pro ukládání dat je využita lehká relační databáze SQLite. Databáze je použita pro ukládání uživatelských účtů a dočasných zpráv. Díky své jednoduchosti je vhodná pro rychlý vývoj. Pro snadnou tvorbu databázového modelu se autor rozhodl použít knihovnu Drizzle ORM.

2.4 Diagram architektury



Obrázek 1 - Diagram architektury aplikace

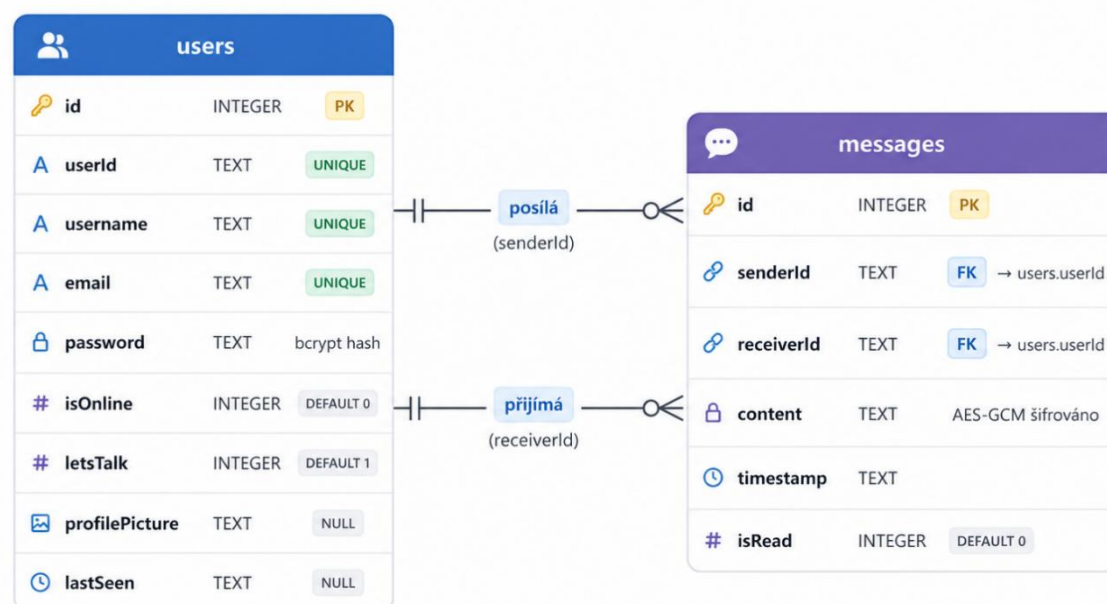
2.4.1 Databázové modely

2.4.1.1 Tabulka uživatelů

- ukládání přihlašovacích údajů a informací o uživateli,
- správa uživatelských statusů,
- validace dat pomocí knihovny Zod.

2.4.1.2 Tabulka zpráv

- dočasné ukládání offline zpráv šifrovaných pomocí AES.



Obrázek 2 - ER model databáze

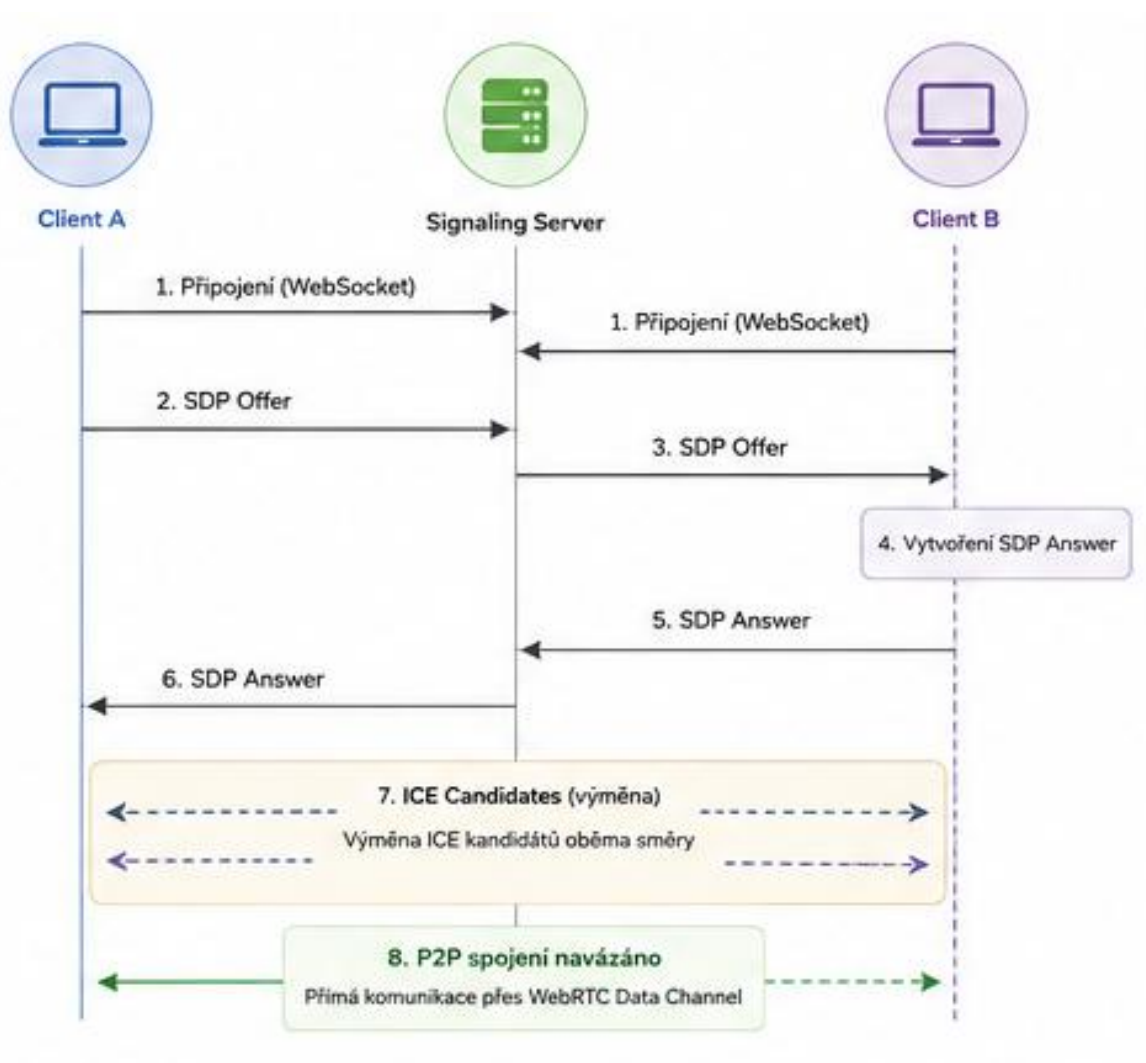
2.5 P2P Komunikace

Pro komunikaci mezi uživateli je využita technologie WebRTC, která umožňuje přímé P2P spojení mezi klienty bez nutnosti přenosu dat přes centrální server. Navázání spojení zajistí signaling mechanismus realizovaný pomocí WebSocket serveru a NAT traversal je řešený s Google STUN serverem a pro komplexnější síť TURN serverem Metered.

2.6 Průběh navázání P2P spojení

Nejprve jsou oba klienti připojeni na backend serveru prostřednictvím WebSocket komunikace. Jakmile se oba připojí do stejné chat konverzace, tak první klient pošle nabídku spojení (SDP offer), která obsahuje informace o možnostech navázání spojení. Tato nabídka je následně odeslána přes signaling server druhému klientovi. Druhý klient nabídku přijme, vytvoří odpověď (SDP answer) a odešle ji zpět prvnímu klientovi.

Během procesu si oba klienti dále vyměňují ICE kandidáty, které obsahují informace o dostupných síťových adresách a možnostech propojení zařízení. Po úspěšné výměně těchto informací dojde k navázání přímého P2P spojení mezi klienty a další komunikace již probíhá bez nutnosti přeposílání dat přes server.



Obrázek 3 - Diagram zahájení P2P spojení

2.6.1 NAT Traversal

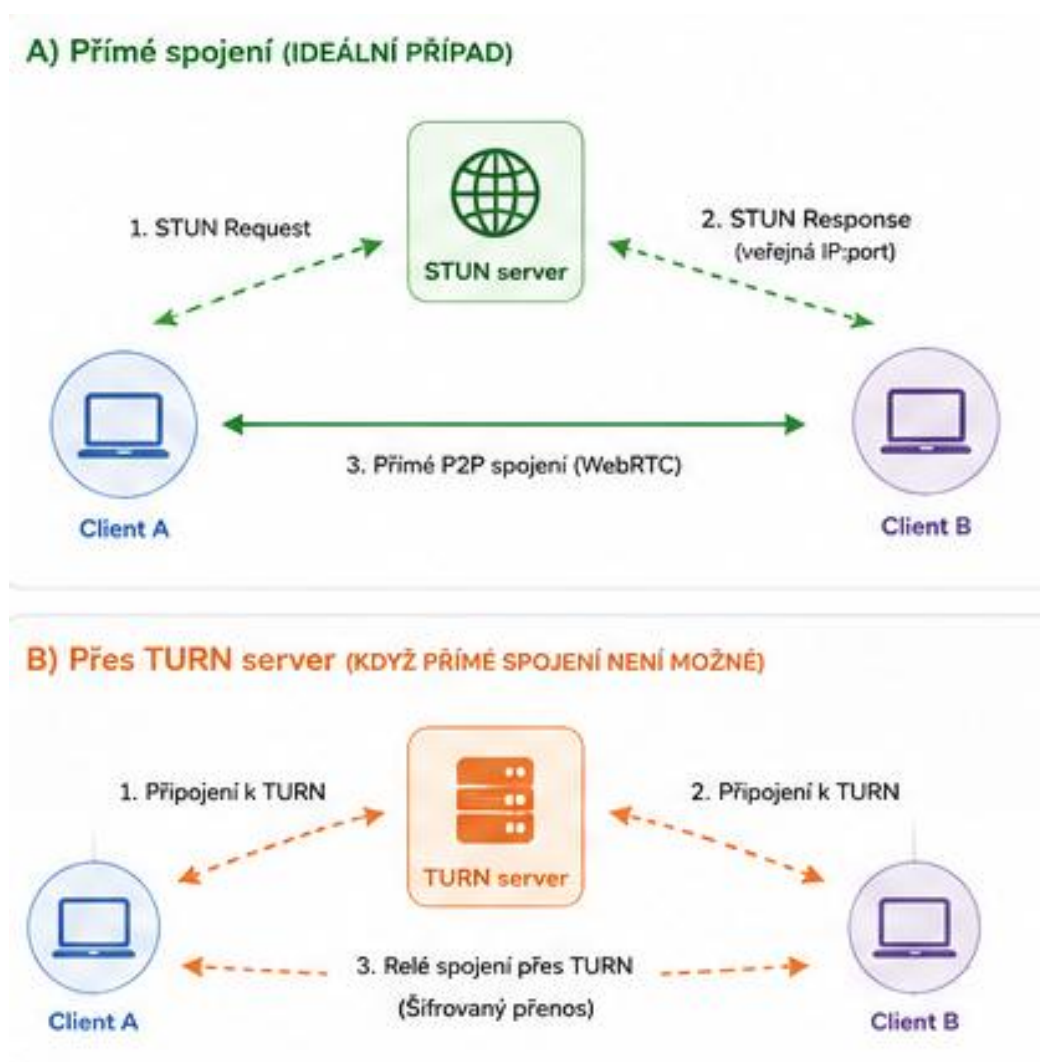
Většina zařízení je v internetu skryta za NAT, která standardně blokuje přímé příchozí spojení. Úspěšné propojení peerů zajistí protokol ICE, který kombinuje dva přístupy.

2.6.1.1 Google STUN

STUN server umožňuje klientovi zjistit svou veřejnou IP adresu a pokusit se o přímé spojení.

2.6.1.2 Metered TURN

Sítě, které jsou za firemními firewally nebo mobilní data většinou přímé spojení nepropustí, proto je třeba využít TURN server, který zajistí hybridní P2P spojení jako zabezpečený prostředník.



Obrázek 4 - Diagram ICE přístupů

3 Požadavky aplikace

3.1 Registrace a přihlášení uživatelů

Aplikace umožňuje registraci pomocí uživatelského jména, e-mailu a hesla. Součástí je také přihlášení a ověření identity. Hesla jsou ukládána v hashované podobě a musí splňovat základní bezpečnostní požadavky.

3.2 Správa uživatelského účtu

Uživatel může upravovat své údaje, například uživatelské jméno nebo profilový obrázek, a zároveň se z aplikace odhlásit.

3.3 Komunikace mezi uživateli

Systém umožňuje vytváření individuálních konverzací mezi uživateli. Uživatelé jsou schopni přijímat i odesílat zprávy v reálném čase. Zároveň je možné zobrazování historie zpráv a psát zprávy i offline uživatelům. Nelze manipulovat s časovou stopou u zpráv. Lze komunikovat přes P2P i když uživatelé využívají mobilní sítě a nenachází se ve stejné síti.

3.4 Bezpečnost komunikace

Aplikace využívá E2EE, kdy jsou zprávy šifrovány na straně klienta a server k nim nemá přístup. Pro šifrování je použit AES-GCM, který zajišťuje důvěrnost i integritu dat.

Komunikace probíhá přes WebRTC se zabezpečením DTLS. Pokud se nepodaří navázat P2P spojení, využije se fallback server, přičemž zprávy zůstávají šifrované.

3.5 Spojení a dostupnost

Aplikace je navržena tak, aby fungovala i při nestabilním připojení. Při výpadku P2P spojení nebo při offline režimu je uživatel informován a zprávy jsou doručeny přes server, i když s nižší úrovní bezpečnosti.

3.6 LetsTalk! funkce

Aplikace umožňuje vysílání statusu LetsTalk!, který signalizuje, že je uživatel otevřený nové konverzaci. Tito uživatelé jsou doporučeni na horní části aplikace.

3.7 Nedávné zprávy

Aplikace filtruje konverzace, které proběhly v blízké minulosti a usnadní uživatelům hledání profilů se kterými komunikují častěji.

3.8 Seznam online uživatelů

Další funkcí aplikace je kompletní seznam všech aktivních i offline uživatelů a možnost s nimi zahájit konverzaci.

3.9 Design a použitelnost

Prostředí je responzivní pro široké spektrum obrazovek a je navrženo i pro běžného uživatele. Lighthouse prohlížeče u analýzy aplikace píše optimální SEO skóre a aplikace splňuje důležité audity funkcionality.

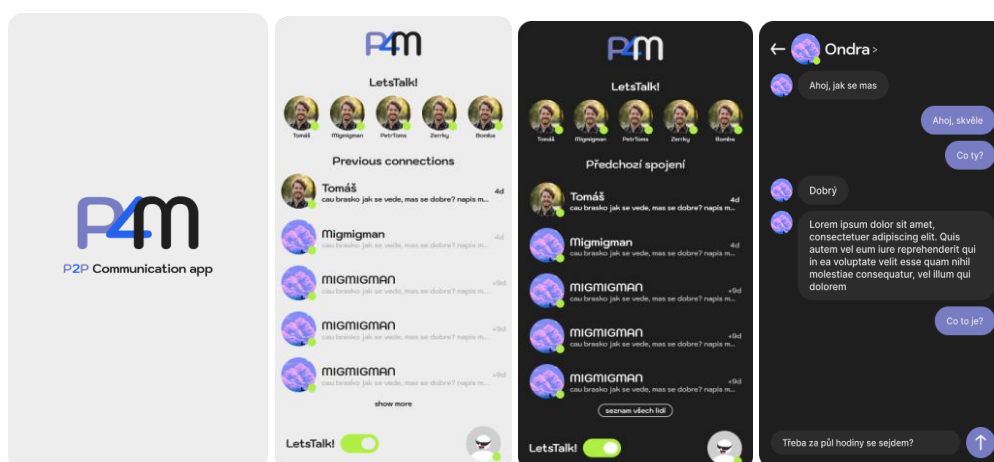
4 Realizace

4.1 Návrh uživatelského rozhraní

Návrh byl vytvořen v nástroji Figma. Tvorba návrhu pomohla u při promyšlení rozložení a vzhledu aplikace, ještě před psaním jakéhokoliv kódu, což přispělo k rychlejší tvorbě designu později u vývoje.

4.1.1 Intuitivní prostředí a identita

Záměrem při návrhu vzhledu aplikace bylo vytvořit jednoduché a moderní uživatelské rozhraní, které bude intuitivní i pro uživatele se zkušenostmi s běžnými komunikačními aplikacemi současnosti. Největší inspirací byla aplikace „Instagram“ s funkcí příběhů a „Discord“ s rozložením UI.



Obrázek 5 - Grafický návrh UI ve Figmě

4.2 Vývoj backendu

Při vývoji backendové části se ukázalo, že čistá P2P architektura by byla z hlediska uživatelského komfortu velmi nepřívětivá. Z tohoto důvodu došlo ke směřování spíše k hybridní P2P aplikaci.

4.2.1 Fallback server

Komunikace přes čisté P2P vyžaduje, aby oba uživatelé byli současně aktivní. To by však znamenalo velké omezení aplikace, a proto bylo rozhodnuto využít WebSocket serveru i pro doručení dočasných zpráv offline uživatelům, než si je zobrazí a uloží se do localStorage.

4.2.2 Zpracování zpráv

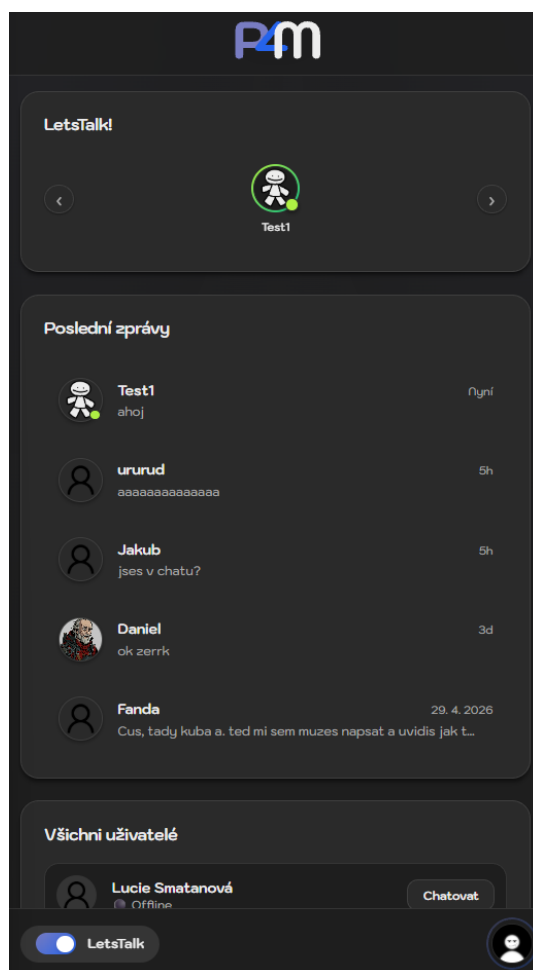
Komunikace serveru a klienta je možná pomocí JSON zpráv s atributem „type“. Podle tohoto atributu server rozhodne, jakým způsobem bude zpráva zpracována.

Mezi podporované typy zpráv patří:

- online – připojení uživatele
- get_users – získání seznamu uživatelů
- signal – WebRTC signalizace
- ice_candidate – přenos ICE kandidátů
- chat_message – textová zpráva mezi uživateli přes fallback server

4.2.3 WebSocket broadcast

Jednou z důležitých součástí backendové části byl WebSocket broadcast pomocí typu zprávy get_users. Každých pár sekund se tak získá stav všech uživatelů a zobrazí se v aplikaci a serverovém logu.



Obrázek 6 - Zobrazení statusu uživatelů v aplikaci

4.3 Vývoj frontendu

4.3.1 Struktura aplikace

Aplikace je rozdělena do jednotlivých komponent. Každá komponenta představuje určitou část uživatelského rozhraní, například navigaci, chatovací okno nebo formuláře pro přihlášení a registraci.

Navigace mezi stránkami je řešena pomocí knihovny React Router. Součástí navigace jsou také tzv. „protected routes“, které v tomto případě zajišťují, že nepřihlášený uživatel nemá přístup ke chráněným částem aplikace. Styly jsou načítány dynamicky pomocí importů `css.modules`. V projektu služby zajišťují komunikaci s databází kontexty sdílení globálního stavu a vlastní hooky propojují tuto logiku s uživatelským rozhraním.

4.3.2 Správa stavů

Sdílení dat napříč celou aplikací řeší technologie React Context API. Tento přístup umožňuje komponentám přistupovat k důležitým datům bez nutného předávání skrze props mezi komponentami.

4.3.2.1 AuthContext

Zajišťuje uchování dat o stavu přihlášení uživatele a poskytuje metody pro přihlášení, registraci a odhlášení.

4.3.2.2 SocketContext

Zajišťuje jediné stabilní WebSocket spojení po celou dobu trvání relace.

4.3.3 Správa konverzací

Aplikace nemá samostatnou tabulku pro ukládání konverzací. Obsah se dynamicky přiřazuje na straně klienta z `localStorage`. Toto opatření zajistí, že kdykoliv lze snadno odstranit konverzace.

4.4 API endpointy

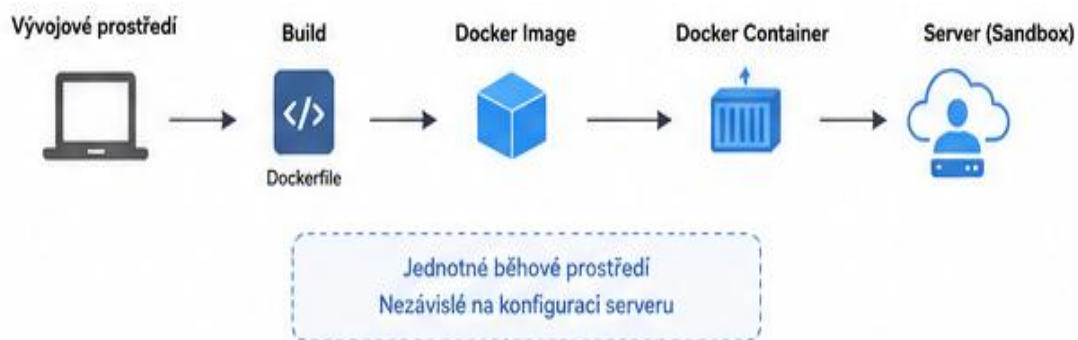
Aplikace využívá několika API endpointů pro komunikaci klienta a serveru:

- GET /health: health check – kontrola funkcionality serveru
- POST /api/auth/register – registrace nového uživatele
- POST /api/auth/login – přihlášení uživatele
- POST /api/auth/logout – odhlášení uživatele

- GET /api/auth/users – získání všech uživatelů
- GET /api/auth/user/:userId – získání uživatele podle id
- POST /api/auth/toggle-letstalk – změna stavu letstalk
- PUT /api/auth/profile – změna dat profilu
- GET /api/messages/history/:user1/:user2 – načtení historie zpráv
- GET /api/messages/conversations/:userId – načtení konverzací

4.5 Nasazení aplikace

Po vytvoření lokálně funkční verze aplikace byla aplikace upravena pro provoz na single-port server a následně zabalena do Docker kontejneru, který zajišťuje jednotné běhové prostředí nezávislé na konfiguraci systému serveru. Díky tomu bylo možné aplikaci snadno nasadit na školní sandbox (<https://id-132.pslib.cloud>) a připravit ji na první testování uživateli. V Dockerfile se i nachází skupina příkazů, které zajistí, že vývojové prostředí zůstane netknuté scriptem a pouze po spuštění pomocné workflow se /dist soubory pošlou na daný server.



Obrázek 7 - Postup nasazení aplikace

5 Testování

Po nasazení aplikace na školní sandbox server byla aplikace zpřístupněna více než 15 testerům za účelem ověření funkčnosti systému v reálném provozu. Testování bylo zaměřeno především na stabilitu aplikace, funkčnost komunikace a celkový uživatelský komfort.

Během testování bylo zjištěno několik nedostatků. U uživatelů bez synchronizovaného systémového času docházelo v některých případech k odesílání duplicitních zpráv. Testerům dále chyběly navigační prvky, například šipky pro jednodušší pohyb mezi jednotlivými částmi aplikace, na které autor při tvorbě nemyslel.

Testování také ukázalo, že P2P spojení nemusí být vždy úspěšně navázáno mezi uživateli nacházejícími se v odlišných sítích za aplikačním firewallem, a komunikace tak v některých případech probíhala pouze přes fallback server.

Získané poznatky pomohly odhalit slabší části aplikace a budou využity při dalším vývoji a optimalizaci systému.

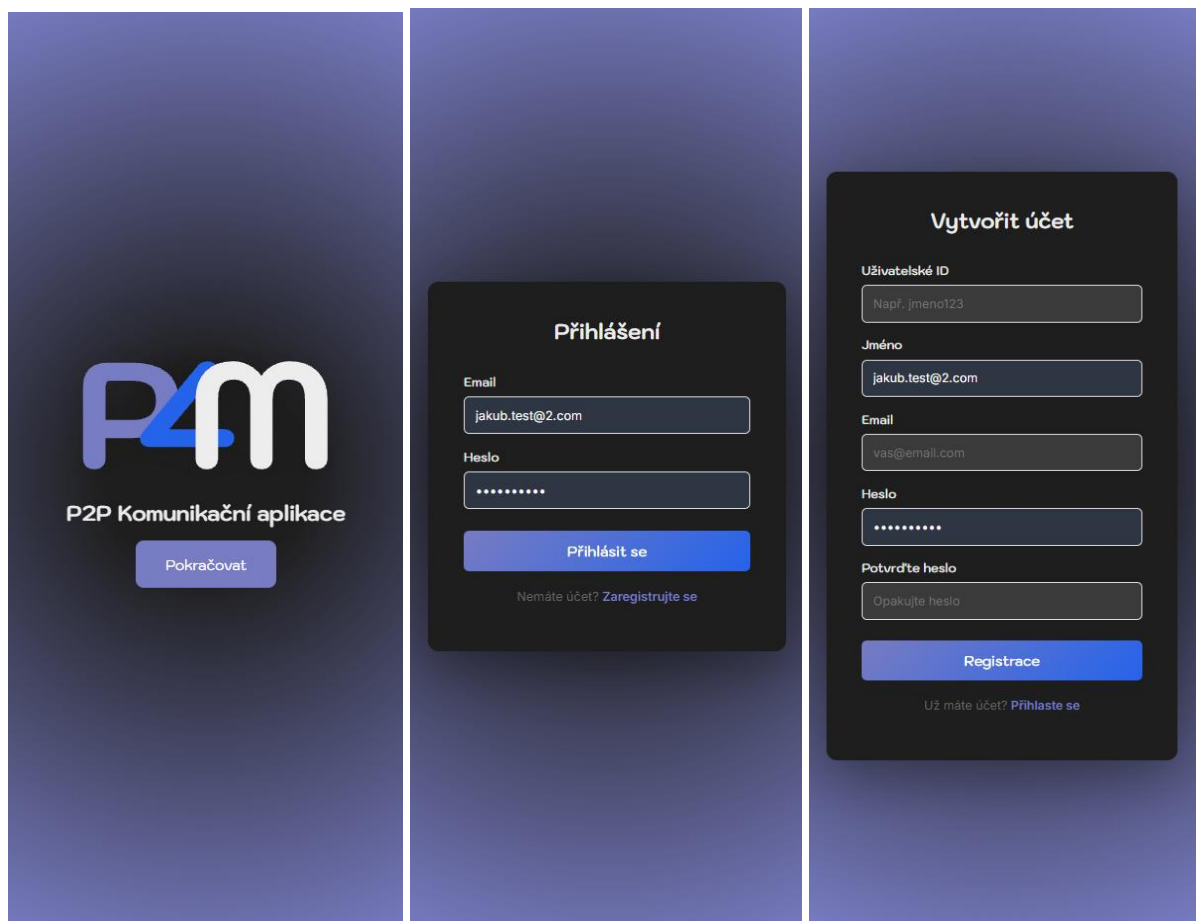
5.1.1 Zapracování zpětné vazby od uživatelů

Po úpravě aplikace na základě zpětné vazby se podařilo odstranit nedostatky zjištěné během testování. Byly doplněny navigační prvky pro jednodušší pohyb v aplikaci a upraven systém zpracování zpráv, aby se minimalizovalo riziko duplicitního odesílání zpráv.

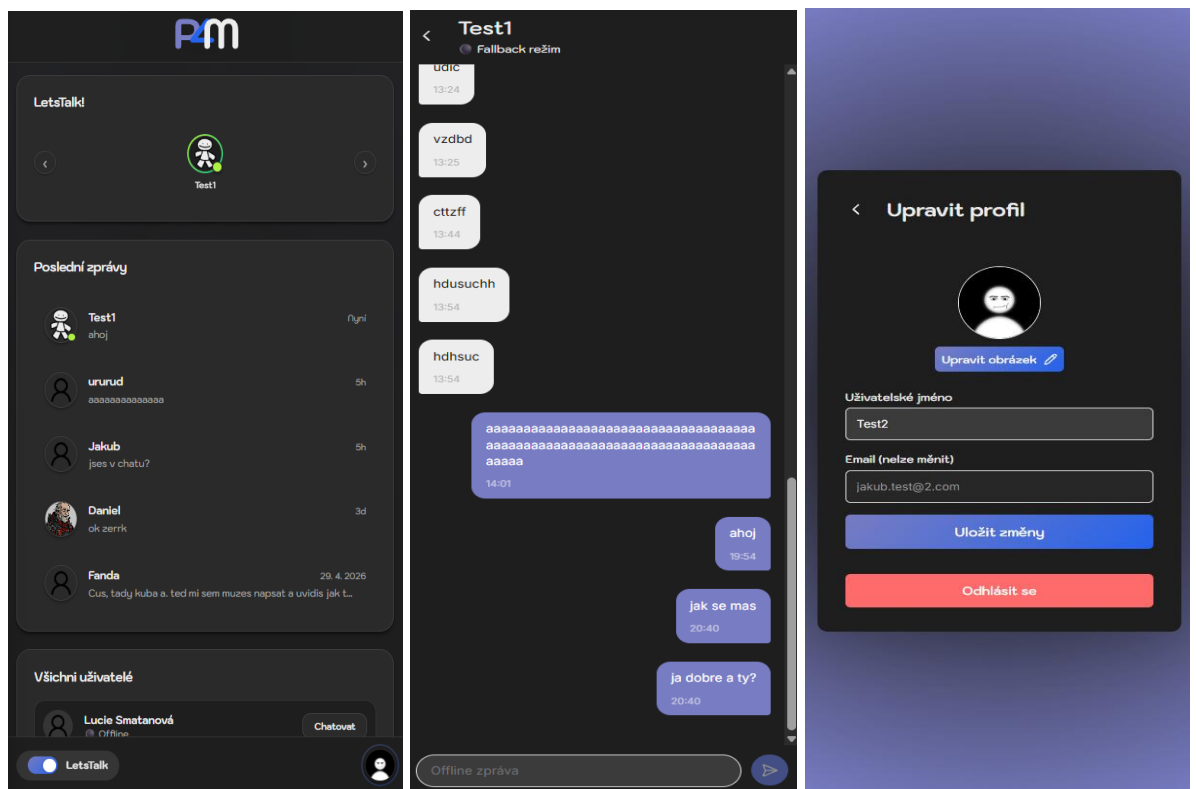
Byla také provedena optimalizace navazování P2P spojení mezi klienty v odlišných sítích pomocí TURN serveru s cílem zvýšit udržitelnost P2P komunikace. Tyto změny přispěly ke zlepšení uživatelského komfortu i celkové stability aplikace.

5.1.1.1 Finální vzhled aplikace

Na základě zpětné vazby od testerů, byla provedena i úprava UI aplikace. Níže jsou uvedeny obrázky zachycující finální podobu aplikace.



Obrázek 8 - Hlavní stránka a autentizační formuláře



Obrázek 9 - Dashboard stránka, chat stránka a stránka na úpravu profilu

Závěr

Cílem této práce bylo navrhnout a implementovat P2P komunikační aplikaci zaměřenou na bezpečnou a efektivní komunikaci mezi uživateli. Během realizace se ukázalo, že čistě P2P přístup naráží na technická omezení, zejména při navazování spojení mezi uživateli v různých síťových prostředích. Z tohoto důvodu bylo řešení rozšířeno o fallback komunikaci prostřednictvím WebSocket serveru, který zajišťuje doručování zpráv i pro offline uživatele.

Součástí práce bylo také uživatelské testování, které odhalilo nedostatky v oblasti uživatelského rozhraní a stability aplikace. Na základě získané zpětné vazby byly tyto problémy následně odstraněny a aplikace byla dále optimalizována s cílem zvýšit její použitelnost a celkovou stabilitu.

Výsledkem práce je funkční komunikační aplikace, která kombinuje principy P2P komunikace s hybridním přístupem pro zvýšení spolehlivosti. Práce zároveň poskytla praktické zkušenosti s vývojem real-time aplikací, implementací serverové části v prostředí Node.js a se základními principy bezpečného přenosu dat v síťových aplikacích.

Seznam zkratek a odborných výrazů

P2P

Peer-to-Peer – způsob komunikace, při kterém si jednotlivá zařízení v síti vyměňují data přímo mezi sebou bez nutnosti centrálního serveru.

UI

User Interface – uživatelské rozhraní, tedy to, co uživatel vidí a s čím přímo interaguje.

UX

User Experience – uživatelský zážitek z používání aplikace nebo systému a jak příjemně a efektivně ho uživatel dokáže ovládat.

API

Application Programming Interface – rozhraní, které umožňuje různým aplikacím mezi sebou komunikovat a vyměňovat data.

Signaling Server

Signaling Server – server používaný v P2P komunikaci, který slouží k navázání spojení mezi dvěma klienty pomocí zprostředkování výměny potřebných informací mezi klienty pro spojení, aniž by sám přenášel samotná data komunikace.

WS

WebSocket – komunikační protokol, který umožňuje obousměrné spojení mezi klientem a serverem.

IP

Internet Protocol – základní síťový protokol, který zajišťuje adresaci a doručení dat mezi zařízeními v počítačové síti pomocí IP adres.

NAT

Network Address Translation – technika, která umožňuje více zařízením v lokální síti sdílet jednu veřejnou IP adresu tím, že překládá jejich interní IP adresy

Frontend

Část aplikace, se kterou přímo pracuje uživatel a zajišťuje vizuální rozhraní a interakci s aplikací v prohlížeči nebo klientovi.

Backend

Serverová část aplikace, která zpracovává logiku, spravuje data, autentizaci a komunikaci mezi frontendem a databází.

Databáze (SQLite)

Systém pro ukládání a správu dat.

Real-time komunikace

Komunikace probíhající v reálném čase, kdy jsou data okamžitě přenášena mezi účastníky.

WebRTC

Technologie umožňující přímou peer-to-peer komunikaci mezi zařízeními (audio, video nebo data)

REST API

Representational State Transfer API – je architektonický styl pro návrh webových rozhraní, který umožňuje komunikaci mezi klientem a serverem pomocí standardních HTTP metod (např. GET, POST, PUT, DELETE).

Endpoint

Konkrétní adresa na serveru, na kterou klient posílá požadavky

ICE

Interactive Connectivity Establishment

SDP

Session Description Protocol

E2EE

End-To-End-Encryption

DTLS

Datagram Transport Layer Security

Seznam obrázků

Obrázek 1 - Diagram architektury aplikace	3
Obrázek 2 - ER model databáze	4
Obrázek 3 - Diagram zahájení P2P spojení.....	5
Obrázek 4 - Diagram ICE přístupů	6
Obrázek 5 - Grafický návrh UI ve Figmě	9
Obrázek 6 - Zobrazení statusu uživatelů v aplikaci.....	10
Obrázek 7 - Postup nasazení aplikace	12
Obrázek 8 - Hlavní stránka a autentizační formuláře	14
Obrázek 9 - Dashboard stránka, chat stránka a stránka na úpravu profilu.....	14

Reference

1. **Platforms, Meta.** *React Documentation*. [Online] 20. 4 2026. <https://react.dev>.
2. **Neznámý.** Sandbox Pslib Cloud nasazení aplikace. *Sandbox Pslib Cloud*. [Online] 20. 4 2026. <https://sandbox.pslib.cloud/applications/132/setup>.
3. **Microsoft.** *TypeScript Documentation*. [Online] 20. 4 2026. <https://www.typescriptlang.org/docs/>.
4. **Foundation, OpenJS.** *Node.js Documentation*. [Online] 20. 4 2026. <https://nodejs.org/en/docs>.
5. **Foundation, Express.js.** *Express.js Documentation*. [Online] 20. 4 2026. <https://expressjs.com/>.
6. **Network, Mozilla Developer.** *WebRTC API*. [Online] 20. 4 2026. https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API.
7. **Mozilla Developer Network.** *WebSocket API*. [Online] 20. 4 2026. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API.
8. **Consortium, SQLite.** *SQLite Documentation*. [Online] 20. 4 2026. <https://www.sqlite.org/docs.html>.
9. **Team, Drizzle.** *Drizzle ORM Documentation*. [Online] 20. 4 2026. <https://orm.drizzle.team/>.
10. **Contributors, Zod.** *Zod Documentation*. [Online] 20. 4 2026. <https://zod.dev/>.
11. **Vite Team.** *Vite Documentation*. [Online] 20. 4 2026. <https://vitejs.dev/guide/>.
12. **Remix Software.** *React Router Documentation*. [Online] 20. 4 2026. <https://reactrouter.com/en/main>.
13. **Docker Inc.** *Docker Documentation*. [Online] 20. 4 2026. <https://docs.docker.com/>.
14. **Lucide Contributors.** *Lucide Icons*. [Online] 20. 4 2026. <https://lucide.dev/>.
15. **OpenAI.** *ChatGPT*. [Online] 14. 5 2026. <https://chatgpt.com/>.
16. **Wikipedia contributors.** Peer-to-peer. *Wikipedia*. [Online] <https://en.wikipedia.org/w/index.php?title=Peer-to-peer&oldid=1354134595>.
17. **Google.** *Gemini 3 (Umělá inteligence)*. [Online] <https://gemini.google.com/>.
18. **Anthropic.** *Claude Sonnet 4.6 AI*. [Online] <https://claude.ai/new>.

A. Seznam příložených souborů

Pouzite_AI_prompty.docx

RP2025-26_Jakub_Adam-P2P_Komunikační_Aplikace.docx

RP2025-26_Jakub_Adam-P2P_Komunikační_aplikace.pdf